In [3]:
```
import pandas as pd

data = {
    'CustomerID': [1, 2, 3, 4, 5, 6, 7, 8, 9, 9, 10],
    'Age_Group': ['20-25', '30-35', '25-30', '20-25', '35+', '35+', '35+', '20-25',
    'Rating(1-5)': [4, 5, 6, -1, 3, 3, 4, 7, 2, 2, 5],
    'Hotel': ['Ibis', 'LemonTree', 'RedFox', 'LemonTree', 'Ibis', 'Ibys', 'RedFox',
    'FoodPreference': ['veg', 'Non-Veg', 'Veg', 'Veg', 'Vegetarian', 'Non-Veg', 'Ve
    'Bill': [1300, 2000, 1322, 1234, 989, 1909, 1000, 2999, 3456, 3456, -6755],
    'NoOfPax': [2, 3, 2, 2, 2, 2, -1, -10, 3, 3, 4],
    'EstimatedSalary': [40000, 59000, 30000, 120000, 45000, 122220, 21122, 345673,
    'Age_Group.1': ['20-25', '30-35', '25-30', '20-25', '35+', '35+', '35+', '20-25
}

df = pd.DataFrame(data)

df.to_csv('Hotel_Dataset.csv', index=False)

print("Hotel_Dataset.csv created successfully!")
```

```
Hotel_Dataset.csv created successfully!
```

In [4]:
```
df = pd.read_csv("Hotel_Dataset.csv")
df
```

Out[4]:

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Est
0	1	20-25	4	Ibis	veg	1300	2	
1	2	30-35	5	LemonTree	Non-Veg	2000	3	
2	3	25-30	6	RedFox	Veg	1322	2	
3	4	20-25	-1	LemonTree	Veg	1234	2	
4	5	35+	3	Ibis	Vegetarian	989	2	
5	6	35+	3	Ibys	Non-Veg	1909	2	
6	7	35+	4	RedFox	Vegetarian	1000	-1	
7	8	20-25	7	LemonTree	Veg	2999	-10	
8	9	25-30	2	Ibis	Non-Veg	3456	3	
9	9	25-30	2	Ibis	Non-Veg	3456	3	
10	10	30-35	5	RedFox	non-Veg	-6755	4	

In [5]:
```
df.duplicated()
```

Out[5]:
```
0     False
1     False
2     False
3     False
4     False
5     False
6     False
7     False
8     False
9      True
10    False
dtype: bool
```

In [6]:
```
df.drop_duplicates(inplace=True)
df
```

Out[6]:

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Est
0	1	20-25	4	Ibis	veg	1300	2	
1	2	30-35	5	LemonTree	Non-Veg	2000	3	
2	3	25-30	6	RedFox	Veg	1322	2	
3	4	20-25	-1	LemonTree	Veg	1234	2	
4	5	35+	3	Ibis	Vegetarian	989	2	
5	6	35+	3	Ibys	Non-Veg	1909	2	
6	7	35+	4	RedFox	Vegetarian	1000	-1	
7	8	20-25	7	LemonTree	Veg	2999	-10	
8	9	25-30	2	Ibis	Non-Veg	3456	3	
10	10	30-35	5	RedFox	non-Veg	-6755	4	

In [7]:
```
len(df)
```

Out[7]:
```
10
```

In [8]:
```
import numpy as np
import pandas as pd
```

In [9]:
```
df = pd.read_csv("Hotel_Dataset.csv")
display(df)
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Estin
0	1	20-25	4	Ibis	veg	1300	2	
1	2	30-35	5	LemonTree	Non-Veg	2000	3	
2	3	25-30	6	RedFox	Veg	1322	2	
3	4	20-25	-1	LemonTree	Veg	1234	2	
4	5	35+	3	Ibis	Vegetarian	989	2	
5	6	35+	3	Ibys	Non-Veg	1909	2	
6	7	35+	4	RedFox	Vegetarian	1000	-1	
7	8	20-25	7	LemonTree	Veg	2999	-10	
8	9	25-30	2	Ibis	Non-Veg	3456	3	
9	9	25-30	2	Ibis	Non-Veg	3456	3	
10	10	30-35	5	RedFox	non-Veg	-6755	4	

In [10]:
```
df.duplicated()
```

Out[10]:
```
0     False
1     False
2     False
3     False
4     False
5     False
6     False
7     False
8     False
9      True
10    False
dtype: bool
```

In [11]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 11 entries, 0 to 10
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerID       11 non-null     int64
 1   Age_Group        11 non-null     str
 2   Rating(1-5)      11 non-null     int64
 3   Hotel            11 non-null     str
 4   FoodPreference   11 non-null     str
 5   Bill             11 non-null     int64
 6   NoOfPax          11 non-null     int64
 7   EstimatedSalary  11 non-null     int64
 8   Age_Group.1      11 non-null     str
dtypes: int64(5), str(4)
memory usage: 924.0 bytes
```

In [12]:
```
df.drop_duplicates(inplace=True)
display(df)
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Estin
0	1	20-25	4	Ibis	veg	1300	2	
1	2	30-35	5	LemonTree	Non-Veg	2000	3	
2	3	25-30	6	RedFox	Veg	1322	2	
3	4	20-25	-1	LemonTree	Veg	1234	2	
4	5	35+	3	Ibis	Vegetarian	989	2	
5	6	35+	3	Ibys	Non-Veg	1909	2	
6	7	35+	4	RedFox	Vegetarian	1000	-1	
7	8	20-25	7	LemonTree	Veg	2999	-10	
8	9	25-30	2	Ibis	Non-Veg	3456	3	
10	10	30-35	5	RedFox	non-Veg	-6755	4	

In [13]:
```
len(df)
```

Out[13]: 10

In [14]:
```
index = np.array(list(range(0, len(df))))
df.set_index(index, inplace=True)
print(index)
display(df)
```

```
[0 1 2 3 4 5 6 7 8 9]
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Estim
0	1	20-25	4	Ibis	veg	1300	2	
1	2	30-35	5	LemonTree	Non-Veg	2000	3	
2	3	25-30	6	RedFox	Veg	1322	2	
3	4	20-25	-1	LemonTree	Veg	1234	2	
4	5	35+	3	Ibis	Vegetarian	989	2	
5	6	35+	3	Ibys	Non-Veg	1909	2	
6	7	35+	4	RedFox	Vegetarian	1000	-1	
7	8	20-25	7	LemonTree	Veg	2999	-10	
8	9	25-30	2	Ibis	Non-Veg	3456	3	
9	10	30-35	5	RedFox	non-Veg	-6755	4	

In [15]:
```
df.drop(['Age_Group.1'], axis=1, inplace=True)
display(df)
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Estim
0	1	20-25	4	Ibis	veg	1300	2	
1	2	30-35	5	LemonTree	Non-Veg	2000	3	
2	3	25-30	6	RedFox	Veg	1322	2	
3	4	20-25	-1	LemonTree	Veg	1234	2	
4	5	35+	3	Ibis	Vegetarian	989	2	
5	6	35+	3	Ibys	Non-Veg	1909	2	
6	7	35+	4	RedFox	Vegetarian	1000	-1	
7	8	20-25	7	LemonTree	Veg	2999	-10	
8	9	25-30	2	Ibis	Non-Veg	3456	3	
9	10	30-35	5	RedFox	non-Veg	-6755	4	

In [16]:
```
df.CustomerID.loc[df.CustomerID < 0] = np.nan
display(df)
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_22484\617225107.py:1: ChainedAssignmentEr
ror: A value is being set on a copy of a DataFrame or Series through chained assignm
ent.
Such chained assignment never works to update the original DataFrame or Series, beca
use the intermediate object on which we are setting values always behaves as a copy
(due to Copy-on-Write).

Try using '.loc[row_indexer, col_indexer] = value' instead, to perform the assignmen
t in a single step.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pan
das-docs/stable/user_guide/copy_on_write.html#chained-assignment
  df.CustomerID.loc[df.CustomerID < 0] = np.nan
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Estim
0	1	20-25	4	Ibis	veg	1300	2	
1	2	30-35	5	LemonTree	Non-Veg	2000	3	
2	3	25-30	6	RedFox	Veg	1322	2	
3	4	20-25	-1	LemonTree	Veg	1234	2	
4	5	35+	3	Ibis	Vegetarian	989	2	
5	6	35+	3	Ibys	Non-Veg	1909	2	
6	7	35+	4	RedFox	Vegetarian	1000	-1	
7	8	20-25	7	LemonTree	Veg	2999	-10	
8	9	25-30	2	Ibis	Non-Veg	3456	3	
9	10	30-35	5	RedFox	non-Veg	-6755	4	

In [17]:
```
df.Bill.loc[df.Bill < 0] = np.nan
display(df)
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_22484\2547988778.py:1: ChainedAssignmentE
rror: A value is being set on a copy of a DataFrame or Series through chained assign
ment.
Such chained assignment never works to update the original DataFrame or Series, beca
use the intermediate object on which we are setting values always behaves as a copy
(due to Copy-on-Write).

Try using '.loc[row_indexer, col_indexer] = value' instead, to perform the assignmen
t in a single step.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pan
das-docs/stable/user_guide/copy_on_write.html#chained-assignment
  df.Bill.loc[df.Bill < 0] = np.nan
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Estim
0	1	20-25	4	Ibis	veg	1300	2	
1	2	30-35	5	LemonTree	Non-Veg	2000	3	
2	3	25-30	6	RedFox	Veg	1322	2	
3	4	20-25	-1	LemonTree	Veg	1234	2	
4	5	35+	3	Ibis	Vegetarian	989	2	
5	6	35+	3	Ibys	Non-Veg	1909	2	
6	7	35+	4	RedFox	Vegetarian	1000	-1	
7	8	20-25	7	LemonTree	Veg	2999	-10	
8	9	25-30	2	Ibis	Non-Veg	3456	3	
9	10	30-35	5	RedFox	non-Veg	-6755	4	

In [18]:
```
df.EstimatedSalary.loc[df.EstimatedSalary < 0] = np.nan
display(df)
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_22484\325803314.py:1: ChainedAssignmentEr
ror: A value is being set on a copy of a DataFrame or Series through chained assignm
ent.
Such chained assignment never works to update the original DataFrame or Series, beca
use the intermediate object on which we are setting values always behaves as a copy
(due to Copy-on-Write).

Try using '.loc[row_indexer, col_indexer] = value' instead, to perform the assignmen
t in a single step.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pan
das-docs/stable/user_guide/copy_on_write.html#chained-assignment
  df.EstimatedSalary.loc[df.EstimatedSalary < 0] = np.nan
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Estim
0	1	20-25	4	Ibis	veg	1300	2	
1	2	30-35	5	LemonTree	Non-Veg	2000	3	
2	3	25-30	6	RedFox	Veg	1322	2	
3	4	20-25	-1	LemonTree	Veg	1234	2	
4	5	35+	3	Ibis	Vegetarian	989	2	
5	6	35+	3	Ibys	Non-Veg	1909	2	
6	7	35+	4	RedFox	Vegetarian	1000	-1	
7	8	20-25	7	LemonTree	Veg	2999	-10	
8	9	25-30	2	Ibis	Non-Veg	3456	3	
9	10	30-35	5	RedFox	non-Veg	-6755	4	

In [19]:
```
df['NoOfPax'].loc[(df['NoOfPax'] < 1) | (df['NoOfPax'] > 20)] = np.nan
display(df)
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_22484\862681127.py:1: ChainedAssignmentEr
ror: A value is being set on a copy of a DataFrame or Series through chained assignm
ent.
Such chained assignment never works to update the original DataFrame or Series, beca
use the intermediate object on which we are setting values always behaves as a copy
(due to Copy-on-Write).

Try using '.loc[row_indexer, col_indexer] = value' instead, to perform the assignmen
t in a single step.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pan
das-docs/stable/user_guide/copy_on_write.html#chained-assignment
  df['NoOfPax'].loc[(df['NoOfPax'] < 1) | (df['NoOfPax'] > 20)] = np.nan
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Estim
0	1	20-25	4	Ibis	veg	1300	2	
1	2	30-35	5	LemonTree	Non-Veg	2000	3	
2	3	25-30	6	RedFox	Veg	1322	2	
3	4	20-25	-1	LemonTree	Veg	1234	2	
4	5	35+	3	Ibis	Vegetarian	989	2	
5	6	35+	3	Ibys	Non-Veg	1909	2	
6	7	35+	4	RedFox	Vegetarian	1000	-1	
7	8	20-25	7	LemonTree	Veg	2999	-10	
8	9	25-30	2	Ibis	Non-Veg	3456	3	
9	10	30-35	5	RedFox	non-Veg	-6755	4	

In [20]:
```
df.Age_Group.unique()
```

Out[20]:
```
<StringArray>
['20-25', '30-35', '25-30', '35+']
Length: 4, dtype: str
```

In [21]:
```
df.Hotel.unique()
```

Out[21]:
```
<StringArray>
['Ibis', 'LemonTree', 'RedFox', 'Ibys']
Length: 4, dtype: str
```

In [22]:
```
df.Hotel.replace(['Ibys'], 'Ibis', inplace=True)
display(df)
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_22484\515593552.py:1: ChainedAssignmentEr
ror: A value is being set on a copy of a DataFrame or Series through chained assignm
ent using an inplace method.
Such inplace method never works to update the original DataFrame or Series, because
the intermediate object on which we are setting values always behaves as a copy (due
to Copy-on-Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method
({col: value}, inplace=True)' instead, to perform the operation inplace on the origi
nal object, or try to avoid an inplace operation using 'df[col] = df[col].method(val
ue)'.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pan
das-docs/stable/user_guide/copy_on_write.html
  df.Hotel.replace(['Ibys'], 'Ibis', inplace=True)
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Estim
0	1	20-25	4	Ibis	veg	1300	2	
1	2	30-35	5	LemonTree	Non-Veg	2000	3	
2	3	25-30	6	RedFox	Veg	1322	2	
3	4	20-25	-1	LemonTree	Veg	1234	2	
4	5	35+	3	Ibis	Vegetarian	989	2	
5	6	35+	3	Ibys	Non-Veg	1909	2	
6	7	35+	4	RedFox	Vegetarian	1000	-1	
7	8	20-25	7	LemonTree	Veg	2999	-10	
8	9	25-30	2	Ibis	Non-Veg	3456	3	
9	10	30-35	5	RedFox	non-Veg	-6755	4	

In [23]:
```
df.FoodPreference.unique()
```

Out[23]:
```
<StringArray>
['veg', 'Non-Veg', 'Veg', 'Vegetarian', 'non-Veg']
Length: 5, dtype: str
```

In [24]:
```
df.FoodPreference.replace(['Vegetarian', 'veg'], 'Veg', inplace=True)
display(df)
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_22484\1951935208.py:1: ChainedAssignmentE
rror: A value is being set on a copy of a DataFrame or Series through chained assign
ment using an inplace method.
Such inplace method never works to update the original DataFrame or Series, because
the intermediate object on which we are setting values always behaves as a copy (due
to Copy-on-Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method
({col: value}, inplace=True)' instead, to perform the operation inplace on the origi
nal object, or try to avoid an inplace operation using 'df[col] = df[col].method(val
ue)'.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pan
das-docs/stable/user_guide/copy_on_write.html
  df.FoodPreference.replace(['Vegetarian', 'veg'], 'Veg', inplace=True)
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Estim
0	1	20-25	4	Ibis	veg	1300	2	
1	2	30-35	5	LemonTree	Non-Veg	2000	3	
2	3	25-30	6	RedFox	Veg	1322	2	
3	4	20-25	-1	LemonTree	Veg	1234	2	
4	5	35+	3	Ibis	Vegetarian	989	2	
5	6	35+	3	Ibys	Non-Veg	1909	2	
6	7	35+	4	RedFox	Vegetarian	1000	-1	
7	8	20-25	7	LemonTree	Veg	2999	-10	
8	9	25-30	2	Ibis	Non-Veg	3456	3	
9	10	30-35	5	RedFox	non-Veg	-6755	4	

In [25]:
```
df.FoodPreference.replace(['non-Veg'], 'Non-Veg', inplace=True)
display(df)
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_22484\2349295012.py:1: ChainedAssignmentE
rror: A value is being set on a copy of a DataFrame or Series through chained assign
ment using an inplace method.
Such inplace method never works to update the original DataFrame or Series, because
the intermediate object on which we are setting values always behaves as a copy (due
to Copy-on-Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method
({col: value}, inplace=True)' instead, to perform the operation inplace on the origi
nal object, or try to avoid an inplace operation using 'df[col] = df[col].method(val
ue)'.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pan
das-docs/stable/user_guide/copy_on_write.html
  df.FoodPreference.replace(['non-Veg'], 'Non-Veg', inplace=True)
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Estim
0	1	20-25	4	Ibis	veg	1300	2	
1	2	30-35	5	LemonTree	Non-Veg	2000	3	
2	3	25-30	6	RedFox	Veg	1322	2	
3	4	20-25	-1	LemonTree	Veg	1234	2	
4	5	35+	3	Ibis	Vegetarian	989	2	
5	6	35+	3	Ibys	Non-Veg	1909	2	
6	7	35+	4	RedFox	Vegetarian	1000	-1	
7	8	20-25	7	LemonTree	Veg	2999	-10	
8	9	25-30	2	Ibis	Non-Veg	3456	3	
9	10	30-35	5	RedFox	non-Veg	-6755	4	

In [26]:
```
df.EstimatedSalary.fillna(round(df.EstimatedSalary.mean()), inplace=True)
display(df)
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_22484\2439528102.py:1: ChainedAssignmentE
rror: A value is being set on a copy of a DataFrame or Series through chained assign
ment using an inplace method.
Such inplace method never works to update the original DataFrame or Series, because
the intermediate object on which we are setting values always behaves as a copy (due
to Copy-on-Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method
({col: value}, inplace=True)' instead, to perform the operation inplace on the origi
nal object, or try to avoid an inplace operation using 'df[col] = df[col].method(val
ue)'.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pan
das-docs/stable/user_guide/copy_on_write.html
  df.EstimatedSalary.fillna(round(df.EstimatedSalary.mean()), inplace=True)
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Estim
0	1	20-25	4	Ibis	veg	1300	2	
1	2	30-35	5	LemonTree	Non-Veg	2000	3	
2	3	25-30	6	RedFox	Veg	1322	2	
3	4	20-25	-1	LemonTree	Veg	1234	2	
4	5	35+	3	Ibis	Vegetarian	989	2	
5	6	35+	3	Ibys	Non-Veg	1909	2	
6	7	35+	4	RedFox	Vegetarian	1000	-1	
7	8	20-25	7	LemonTree	Veg	2999	-10	
8	9	25-30	2	Ibis	Non-Veg	3456	3	
9	10	30-35	5	RedFox	non-Veg	-6755	4	

In [27]:
```
df.NoOfPax.fillna(round(df.NoOfPax.median()), inplace=True)
display(df)
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_22484\3209154726.py:1: ChainedAssignmentE
rror: A value is being set on a copy of a DataFrame or Series through chained assign
ment using an inplace method.
Such inplace method never works to update the original DataFrame or Series, because
the intermediate object on which we are setting values always behaves as a copy (due
to Copy-on-Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method
({col: value}, inplace=True)' instead, to perform the operation inplace on the origi
nal object, or try to avoid an inplace operation using 'df[col] = df[col].method(val
ue)'.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pan
das-docs/stable/user_guide/copy_on_write.html
  df.NoOfPax.fillna(round(df.NoOfPax.median()), inplace=True)
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Estim
0	1	20-25	4	Ibis	veg	1300	2	
1	2	30-35	5	LemonTree	Non-Veg	2000	3	
2	3	25-30	6	RedFox	Veg	1322	2	
3	4	20-25	-1	LemonTree	Veg	1234	2	
4	5	35+	3	Ibis	Vegetarian	989	2	
5	6	35+	3	Ibys	Non-Veg	1909	2	
6	7	35+	4	RedFox	Vegetarian	1000	-1	
7	8	20-25	7	LemonTree	Veg	2999	-10	
8	9	25-30	2	Ibis	Non-Veg	3456	3	
9	10	30-35	5	RedFox	non-Veg	-6755	4	

In [28]:
```
df['Rating(1-5)'].fillna(
    round(df['Rating(1-5)'].median()),
    inplace=True
)
display(df)
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_22484\432956862.py:1: ChainedAssignmentEr
ror: A value is being set on a copy of a DataFrame or Series through chained assignm
ent using an inplace method.
Such inplace method never works to update the original DataFrame or Series, because
the intermediate object on which we are setting values always behaves as a copy (due
to Copy-on-Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method
({col: value}, inplace=True)' instead, to perform the operation inplace on the origi
nal object, or try to avoid an inplace operation using 'df[col] = df[col].method(val
ue)'.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pan
das-docs/stable/user_guide/copy_on_write.html
  df['Rating(1-5)'].fillna(
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Estim
0	1	20-25	4	Ibis	veg	1300	2	
1	2	30-35	5	LemonTree	Non-Veg	2000	3	
2	3	25-30	6	RedFox	Veg	1322	2	
3	4	20-25	-1	LemonTree	Veg	1234	2	
4	5	35+	3	Ibis	Vegetarian	989	2	
5	6	35+	3	Ibys	Non-Veg	1909	2	
6	7	35+	4	RedFox	Vegetarian	1000	-1	
7	8	20-25	7	LemonTree	Veg	2999	-10	
8	9	25-30	2	Ibis	Non-Veg	3456	3	
9	10	30-35	5	RedFox	non-Veg	-6755	4	

In [29]:
```
df.Bill.fillna(round(df.Bill.mean()), inplace=True)
display(df)
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_22484\3902675100.py:1: ChainedAssignmentE
rror: A value is being set on a copy of a DataFrame or Series through chained assign
ment using an inplace method.
Such inplace method never works to update the original DataFrame or Series, because
the intermediate object on which we are setting values always behaves as a copy (due
to Copy-on-Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method
({col: value}, inplace=True)' instead, to perform the operation inplace on the origi
nal object, or try to avoid an inplace operation using 'df[col] = df[col].method(val
ue)'.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pan
das-docs/stable/user_guide/copy_on_write.html
  df.Bill.fillna(round(df.Bill.mean()), inplace=True)
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Estim
0	1	20-25	4	Ibis	veg	1300	2	
1	2	30-35	5	LemonTree	Non-Veg	2000	3	
2	3	25-30	6	RedFox	Veg	1322	2	
3	4	20-25	-1	LemonTree	Veg	1234	2	
4	5	35+	3	Ibis	Vegetarian	989	2	
5	6	35+	3	Ibys	Non-Veg	1909	2	
6	7	35+	4	RedFox	Vegetarian	1000	-1	
7	8	20-25	7	LemonTree	Veg	2999	-10	
8	9	25-30	2	Ibis	Non-Veg	3456	3	
9	10	30-35	5	RedFox	non-Veg	-6755	4	

In [ ]: